

CAN Simulator GUI

Dokumentacja projektu

Dominik Maślak

20 kwietnia 2026

Spis treści

1	Wprowadzenie	2
2	Główne funkcje	2
3	Wymagania	3
4	Szybki start	3
4.1	Klonowanie repozytorium	3
4.2	Utworzenie wirtualnego środowiska i instalacja zależności	3
4.3	Uruchomienie aplikacji	3
4.4	Uruchomienie testów jednostkowych (opcjonalnie)	3
5	Opis zakładek	3
6	Tryb CLI	3
7	Testy jednostkowe	4
8	Struktura projektu	4
9	Rozwój i wkład	4
10	Licencja	4

1 Wprowadzenie

CAN Simulator GUI to zaawansowane narzędzie do analizy logów magistrali CAN, symulacji brakujących modułów oraz inteligentnego wyszukiwania ramek odpowiedzialnych za pojawianie się i zanikanie alertów (np. kontrolki błędów na desce rozdzielczej).

Aplikacja łączy:

- interaktywne wyszukiwanie binarne,
- uczenie maszynowe (regresja logistyczna),
- opcjonalne uczenie przez wzmocnienie (Q-learning).

Dzięki temu znacząco przyspiesza proces diagnostyki magistrali CAN.

2 Główne funkcje

- **Odtwarzanie logów CAN** – wczytywanie plików w formacie `candump` i odtwarzanie z zadanim interwałem oraz przyspieszeniem.
- **Tryb krokowy** – ręczne wysyłanie kolejnych ramek z podglądem.
- **Symulacja brakującego modułu** – cykliczne wysyłanie zestawu ramek (m.in. 0C00008F, 1CFF66F0) symulujących pracę modułu.
- **Generowanie ramek błędów** – sekwencje ramek 0C000005 i 14200032 symulujące awarię.
- **Ręczne wysyłanie ramek** – wsparcie dla ramek standardowych i rozszerzonych (29-bit), z opcją cyklicznego wysyłania.
- **Wyszukiwanie binarne** – interaktywne znajdowanie ramki powodującej wystąpienie (lub zanik) zjawiska.
 - Tryb ręcznego podziału na części.
 - Automatyczne polowanie na dezaktywator alertu (detekcja zaniku cyklicznej ramki).
- **Uczenie maszynowe** – model regresji logistycznej sugeruje optymalne punkty podziału i kolejność przeszukiwania.
- **Uczenie przez wzmocnienie (RL)** – agent Q-learning dynamicznie dostosowuje punkt startowy polowania, aby skrócić czas poszukiwań.
- **Analiza sekwencji** – wykrywanie powtarzających się sekwencji ramek w oknie przed dezaktywacją.
- **Tryb CLI** – możliwość uruchomienia wyszukiwania binarnego z linii poleceń (bez GUI).
- **Tryb offline** – praca bez fizycznego adaptera CAN (symulacja wysyłania).

3 Wymagania

- Python 3.6+
- System Linux z obsługą SocketCAN (do pracy z rzeczywistą magistralą) – opcjonalnie
- Zależności Pythona (patrz plik `requirements.txt`)

4 Szybki start

4.1 Klonowanie repozytorium

```
1 git clone https://github.com/dominikmaslak11/can_simulator.git
2 cd can_simulator
```

4.2 Utworzenie wirtualnego środowiska i instalacja zależności

```
1 python3 -m venv venv
2 source venv/bin/activate
3 pip install --upgrade pip
4 pip install -r requirements.txt
```

4.3 Uruchomienie aplikacji

```
1 ./run.sh
```

lub ręcznie:

```
1 source venv/bin/activate
2 python3 main.py
```

4.4 Uruchomienie testów jednostkowych (opcjonalnie)

```
1 pytest
```

5 Opis zakładek

6 Tryb CLI

Wyszukiwanie binarne można uruchomić z linii poleceń, np.:

```
1 python cli.py --file candump.log --mode find_start --interval 0.1
```

Argumenty:

- `-file` – ścieżka do pliku z logiem (wymagane)
- `-interval` – interwał odtwarzania [s] (domyślnie 0.1)

- `-mode` – `find_start` (początek zjawiska) lub `find_end` (koniec)
- `-start` – indeks startowy (domyślnie 0)
- `-end` – indeks końcowy (domyślnie ostatni)

7 Testy jednostkowe

Projekt zawiera testy `pytest` dla krytycznych komponentów (`parsers.py`, `CyclicDetector`).
Uruchom je poleceniem:

```
1 pytest
```

8 Struktura projektu

```
can_simulator/  
  main.py                # Punkt wejścia GUI  
  cli.py                 # Tryb CLI  
  can_interface.py       # Abstrakcyjna klasa interfejsu CAN  
  socketcan_interface.py # Implementacja SocketCAN  
  dummy_interface.py     # Interfejs offline  
  parsers.py             # Parsowanie logów candump  
  cyclic_detector.py     # Detekcja cykliczności alertów  
  sequence_analyzer.py   # Analiza powtarzających się sekwencji  
  rl_agent.py            # Agent Q-learning  
  ml/                   # Moduł uczenia maszynowego  
  threads/              # Wątki symulacji i wyszukiwania  
  gui/  
    app.py              # Interfejs graficzny  
    handlers.py  
    simulation_handlers.py  
    binary_handlers.py  
    tabs/              # Definicje zakładek  
  tests/               # Testy jednostkowe  
  requirements.txt  
  run.sh               # Skrypt uruchomieniowy (aktywuje venv)  
  README.md
```

9 Rozwój i wkład

Zachęcam do zgłaszania błędów i propozycji funkcji przez **Issues** na GitHub. Pull requesty są mile widziane.

10 Licencja

Projekt dostępny na licencji **MIT** – szczegóły w pliku `LICENSE`.

Zakładka	Opis
Odtwarzanie pliku	Wczytanie logu i odtworzenie z zadany�m interwa�em.
Symulacja modu�u	Cykliczne wysy�anie ramek brakuj�cego modu�u (konfigurowalne interwa�y).
Ramki b�edu	Generowanie sekwencji ramek b�ed�w.
Tryb krokowy	R�czne wysy�anie ramek jedna po drugiej.
Wysy�anie r�czne	Definiowanie i wysy�anie pojedynczych/cyklicznych ramek.
Wyszukiwanie binarne	G�w�ne centrum poszukiwa� – wyb�r trybu: pocz�tek/koniec zjawiska, r�czny podzia�, automatyczne polowanie, polowanie z RL.

Tabela 1: Opis zak adek aplikacji